Explanation of Errors in Version 1

Confusion around the custom AIFM model used, and how it was used to produce reported results

The custom version of the AIFM I linked in the Version 1 document, "aifm-taste-singularity", had different custom changes than I thought it did. I have quite a few custom versions of the AIFM lying around and unfortunately I got mixed up putting them together. The linked version has three main changes:

1. 14 bug fixes for the AIFM, 1 of which I've verified, the rest of which were previously vibecoded between Fable 5.1, Opus 5.5, and GPT-6 Astra.
2. A change to hold β fixed when increasing the Infinite Coding Labor Asymptote, instead of allowing calibration effects to change it, to match my claim that the Asymptote is ~fine at 15x currently but will rise over time.
3. A change that added flags for controlling exactly where λ is increased in the AIFM. (ex. can set λ to 0.65 overall, but keep it at 0.5 when dealing with the Coding Automation Efficiency Improvement Factor (η); I had this code around in order to calculate timeline effects from every use of λ in the AIFM.)

I *thought* part-way through creating my Version 1 document that instead it had changes:

A. Adjusted handling of λ so that calibration effects don't turn higher λ into longer timelines, as performed in "aifm-lambda-fixes", which was a core output of my LW post on how the AIFM works with λ. (false, wasn't in there)
B. A change to hold β fixed (true, same thing as 2 above)

The core p(taste-only) Monte Carlo simulation was done with just the bug fixes (1) and β-hold changes (2) in effect. After realizing that the adjusted handling of λ (A) was missing, I attempted to add it back in. However I instead added in the fine-grained flag-based λ handling (3). I did not realize this error until after submission.

I then used that updated version of the code (1,2,3) to calculate how much takeoff is shortened by adjusting the parameters as described, but *not* to redo Monte Carlo for p(taste-only) (which used just 1,2), due to time constraints. That meant that my p(taste-only) calculations actually still included calibration effects on β from changing λ. The consequence was slightly higher β and slightly lower *m*, which decreased p(taste-only) by about 2 percentage points across all three values of *w* compared to what I would have calculated with the "aifm-lambda-fixes" changes.

I've cut the numbers on shortened takeoffs from Version 2 as irrelevant. But for completeness here, with the updated "aifm-taste-singularity-revamped" code, I find that increasing λ from 0.6 to 0.9 only shortens takeoff by 1 to 3%. (The ~10-20% numbers from my original document version are partly because I varied from 0.5 to 0.9, and mostly because I ran the code with flags that only applied λ to future coding labor growth, and not to its use with η or with the Maximum

Serial Coding Labor Multiplier.) I also find that increasing the Asymptote from 10x to 75x makes for a 40% faster takeoff; but the 15x to 50x effect described in Version 1 remains ~20-30%, as the code for handling the Asymptote remains the same. (This considerable difference between the effects of λ and the Asymptote reflects my findings in my LW post here that the Asymptote is much more important during takeoff than λ; I should have realized that my numbers in the original post didn't make sense, given that.)

However, to be clear, the code in the "aifm-taste-singularity" repo *can* replicate all the numbers I provided in the original document. It was the code I used for them. To replicate my calculations, see the appendix below (written by Claude).

Incomplete AI usage disclosure

I forgot to mention that I used Fable 5.1 and Opus 5.5 to create the new custom AIFM repo, and that GPT-6 Astra was used alongside them to create the original custom AIFM repos I stitched together to make it. Hopefully that was obvious considering the repo includes Claude coauthorship, and considering my linked LW post explains my AI usage in creating the original repos, but I should have been explicit in the disclaimer.

Mistaken epistemic status disclosure

I said the new custom AIFM was based on "a version I'm confident in", by which I meant the "aifm-lambda-fixes" version, but as discussed above it turns out I hadn't actually based the new custom AIFM on that version at all.

Appendix: Replicating the Version 1 calculations (written by Claude)

All Version 1 numbers come from JulianBradshaw/aifm-taste-singularity at commit 5be72ed. The Monte Carlo runs use changes 1 and 2 (the bug fixes and the β-hold), with λ fixed at 0.65 for history and the future alike. The deterministic λ runs behind Version 1's ~10–20% takeoff figure add change 3 (the λ-sites flags).

Setup

```
git clone https://github.com/JulianBradshaw/aifm-taste-singularity
cd aifm-taste-singularity
git checkout 5be72ed
python3.14 -m venv .venv
.venv/bin/python -m pip install -r requirements.txt
cd taste_singularity
export AIFM_ROOT=..
```

(On Windows: `.venv\Scripts\python.exe` and `set AIFM_ROOT=..`. From here on, `python` means the venv's python.)

Monte Carlo (Eli Q2 sampling config, λ 0.65, 300 samples, seed 20260810)

```
python mc.py 300 mc300.jsonl          # scenarios a (default), b (TasteVal), c (w = 0.3/0.5/0.7),
                                        d/e (50x, unbounded, β refit)
python mcfix.py 300 mcfix.jsonl       # 50x / unbounded with β held per sample at its 15x value
python mcfix2.py 300 mcfix2.jsonl     # the same with m held too
cat mc300.jsonl mcfix.jsonl mcfix2.jsonl > all.jsonl
python ana.py all.jsonl               # P(TOS), median m/β, AC→ASI, ASI, P(ASI < 2030) per scenario
python mc2.py 300 mc2_300.jsonl       # grid λ {0.65, 0.9} × ceiling {15x, 50x} × w {0.3, 0.5, 0.7},
                                        β and m held
python ana2.py                        # the grid summary (P(TOS) 71 / 65 / 55% by w)
```

mc.py takes about 20 minutes on 4 cores, and mc2.py about 30. Every mc2.py grid cell holds β and m per sample at their 15x values, so P(TOS) is flat across λ and the Asymptote by construction. Version 1's ~20–30% faster takeoff for an Asymptote of 50x instead of 15x is the difference in median AC→ASI between the 15x and 50x cells of this grid.

λ-flag deterministic runs (the runs behind Version 1's ~10–20% λ figure)

Setting: `parallel_penalty = 0.5` (history and calibration at 0.5), `LAM_SITES = {"dynamics": λ}` for λ 0.65 and 0.9. Every other site and η stay at 0.5. λ 0.5 uses no sites. The taste slope is scaled and iterated until m hits w·0.24 + (1−w)·0.615. β is held with `AIFM_FIX_R=3.1377461411776673`, the λ 0.5, 15x value. lambda_sites.py sets the LAM_* variables itself.

Save this as `v1_lambda.py` in `taste_singularity/` and run it with `AIFM_FIX_R=3.1377461411776673 python v1_lambda.py`:

```
import lambda_sites as L
MD, MS = 0.24, 0.615
p0 = dict(L.preset('eli-08-16-26'), parallel_penalty=0.5)
def run(w, lam, asym):
    mt = w*MD + (1-w)*MS; s = 2.1*mt/0.2450743507356192
    for _ in range(4):
        p = dict(p0, ai_research_taste_slope=s, inf_labor_asymptote=asym)
        o = L.model_run(p, {} if lam == 0.5 else {'dynamics': lam}, detail=True)
        m = o['model'].results['slope_times_log_f']
        if abs(m - mt) < 1e-4: break
        s *= mt/m
    return o['asi'] - o['ac']
for asym in (15.142135623730951, 50.0):
    for w in (0.3, 0.5, 0.7):
        print(int(asym), w, ' '.join(f"λ{l}: {run(w, l, asym):.2f} yr" for l in (0.5, 0.65, 0.9)))
```

Check: at 15x and w = 0.5 the script prints 0.94 / 0.83 / 0.74 yr for λ 0.5 / 0.65 / 0.9. At 15x, the takeoff speed-ups across w are 11–13% for λ 0.5 → 0.65 and 19–23% for λ 0.5 → 0.9, which is the source of the ~10–20% figure. The script's full output (AC→ASI, years, for λ 0.5 / 0.65 / 0.9), reproduced on 2026-11-02:

Ceiling	**w = 0.3**	**w = 0.5**	**w = 0.7**
15x	0.78 / 0.68 / 0.60	0.94 / 0.83 / 0.74	1.23 / 1.10 / 1.00
50x	0.62 / 0.52 / 0.41	0.73 / 0.61 / 0.49	0.92 / 0.77 / 0.64

The 15x rows come out the same with or without AIFM_FIX_R. The 50x rows need it.